Machine Learning for Physicists

A Nutshell Guide to the Practical Man

Caio Laganá Fernandes

Preface

So, you're a physicist looking to dive into the world of machine learning. You have a deep understanding of calculus, are familiar with experimental data-fitting methods, and have mastered concepts like entropy, non-linearity and hypersurface—yet you have only a vague idea of how a machine learning algorithm is actually written and how it works? Then this book is for you. And the good news is: your learning curve will grow fast. I've been in your shoes. I used to view machine learning as a distant horizon, but I soon realized that the knowledge I'd gained in physics made it incredibly easy to quickly master the concepts and engineering behind the field. So, come along and be amazed.

Chapter 1

One-neuron ML

You have been using a single-neuron network since the first time you fitted experimental data, but perhaps no one ever told you that. Let's recall the basics. You have a set of N experimental data points

$$(x_i, y_i) \tag{1.1}$$

through which to fit a linear function

$$f(x) = ax + b. \tag{1.2}$$

The Mean Squared Error Loss, defined as

$$\begin{aligned} L_{\text{MSE}}(a, b) &= \frac{1}{N}\sum_{i=1}^{N}[y_i - f(x_i)]^2 \\ &= \frac{1}{N}\sum_{i=1}^{N}[y_i - (ax_i + b)]^2 \end{aligned} \tag{1.3}$$

tells us how distant the curve $f(x)$ is from the datapoints. Fitting $f(x)$ to the datapoints (x_i, y_i) means minimizing $L_{\text{MSE}}(a, b)$ with respect to a and b, that is,

$$\frac{\partial}{\partial(a, b)} L_{\text{MSE}}(a, b) = 0. \tag{1.4}$$

No surprises so far, right? Well, what if I tell you the above is the simplest possible neural network? Indeed, it is a single-neuron network, where (1.2) are the model's weights, (1.3) is the loss function and (1.4) is the embryo of the gradient descent method.

Solving (1.4) for a and b, which, in this simple case is possible analytically, fixes the parameters to a_0 and b_0

$$\begin{aligned} a_0 &= \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} \\ b_0 &= \bar{y} - a_0 \bar{x} \end{aligned} \tag{1.5}$$

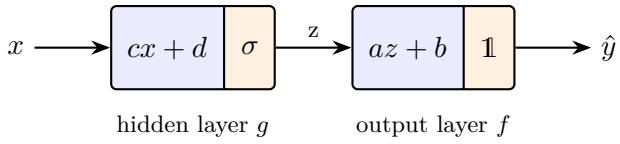
and we end up with a model $f_0(x) = a_0 x + b_0$ (in (1.5), $\bar{x}$ and $\bar{y}$ are the mean values of the x_i and y_i datapoints). This model is a single neuron, no hidden layers: it receives one input parameter and outputs a number. f_0 has learned from data (1.1) through (1.4) and can now predict on any given x. Pretty cool, huh? The concepts were sitting there all the time. Take a moment to re-read the definitions and let's jump to the second simplest model: a two-neuron network.

Chapter 2

Two-neuron ML

Our neuron from Chap. 1 was of the form $f(x) = ax + b$ (always think of it as receiving one value x and, upon training for a, b parameters, outputting another value $a_0 x + b_0$). Let us now place a second neuron, g, before it, so that g receives the input and f produces the final output. This new neuron will be of the form $g(x) = \sigma(cx + d)$, where $\sigma(x) = 1/(1 + e^{-x})$. The need for σ will be elucidated later; for now, just accept it as a magical ingredient. In fact, it is the neuron's so-called activation function!

Placing one neuron before another means composing the two functions, that is, given input x, the network outputs $f(g(x))$. In other words, input x feeds a (hidden) layer g, which applies its weight c and bias d, activates through σ and feeds output layer f, which in turn applies weight a and bias b and outputs $\hat{y}$. Layer f has identity operator $\mathbb{1}$ as its activation function. A visual diagram may help.



Our task now is to optimize the network for parameters a, b, c, d over training data. The loss function is similar to (1.3), just generalized for the two-neuron case,

$$L_{\text{MSE}}(a, b, c, d) = \frac{1}{N} \sum_{i=1}^{N} [y_i - f(g(x_i))]^2. \tag{2.1}$$

This time, however, MSE minimization, stated as

$$\frac{\partial}{\partial(a, b, c, d)} L_{\text{MSE}}(a, b, c, d) = 0 \tag{2.2}$$

cannot be solved analytically as in (1.5), and numerical methods must be used. The recipe is as follows: start at a certain initial position (a, b, c, d) in the four-dimensional parameter space,

then slightly change each coordinate by an amount according to[1]:

$$\begin{aligned} a &\to a - \alpha \frac{\partial L_{\text{MSE}}}{\partial a} \\ b &\to b - \alpha \frac{\partial L_{\text{MSE}}}{\partial b} \\ c &\to c - \alpha \frac{\partial L_{\text{MSE}}}{\partial c} \\ d &\to d - \alpha \frac{\partial L_{\text{MSE}}}{\partial d} \end{aligned} \tag{2.3}$$

The term α is called the learning rate (hallelujah, a name you've heard before, now in a clear context!). The derivatives of L_{MSE} in (2.3) point to the max uphill on the hypersurface, so the minus sign will move the initial coordinate to a new position where L_{MSE} is lower. After repeating this procedure a couple of times, (2.2) will be solved to a certain desired precision (yes, I am oversimplifying: this step is a billion-dollar industry).

The recipe just described for adjusting (a, b, c, d) parameters based on the partial derivatives (2.3) is called *gradient descent*. The computation of the derivatives themselves will lead to the concept of *backpropagation*. Stated like this, with bare partial derivatives and almost no intuition of what is going on behind the scenes, the recipe sounds a bit barren. Let me improve it.

The central idea of gradient descent and backpropagation is to tell (or, more precisely, to *signal*) the neurons in which direction they must adjust their parameters, and by how much, in order to minimize the final loss. Think about it: how can a hidden neuron (in our two-neuron model, this is g) possibly gather information about the loss at the very output? This is the central problem to be addressed. Let's dive a little into the algebra.

The partial derivatives present in (2.3), obtained through chain-rule derivatives on (2.1), can, in this case, be taken explicitly:

$$\begin{aligned} \frac{\partial L}{\partial a} &= \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial a} = -\frac{2}{N} \sum_i \left[y_i - f(g(x_i)) \right] g(x_i) \\ \frac{\partial L}{\partial b} &= \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial b} = -\frac{2}{N} \sum_i \left[y_i - f(g(x_i)) \right] \\ \frac{\partial L}{\partial c} &= \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial c} = -\frac{2a}{N} \sum_i \left[y_i - f(g(x_i)) \right] \sigma'(cx_i + d) x_i \\ \frac{\partial L}{\partial d} &= \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial d} = -\frac{2a}{N} \sum_i \left[y_i - f(g(x_i)) \right] \sigma'(cx_i + d) \end{aligned} \tag{2.4}$$

A suitable reparametrization of variables will be of great algebraic assistance. (We know from other branches of physics—electromagnetism, condensed matter, particle physics—that a "suitable change of variables" often carries deep phenomenological significance. So, hold on!)

$$\begin{aligned} x_i &\quad \text{input} \\ u_i = cx_i + d &\quad \text{hidden pre-activation} \\ z_i = \sigma(u_i) &\quad \text{hidden activation} \\ \hat{y}_i = f(g(x_i)) = az_i + b &\quad \text{network output} \end{aligned} \tag{2.5}$$

In terms of (2.5), define the *error signal* as

$$\delta_i = \frac{\partial L}{\partial \hat{y}_i} = -\frac{2}{N}(y_i - \hat{y}_i), \tag{2.6}$$

[1] Picture this process as finding a minimum on a graph. For reference, in Chap. 1 the graph of the loss function $(a, b, L_{\text{MSE}}(a, b))$ was a two-dimensional paraboloid embedded in $\mathbb{R}^3$; in this two-neuron scenario, the graph is a four-dimensional hypersurface embedded in $\mathbb{R}^5$. The paraboloid has a clear, well-defined minimum; this time, there may be several local minima plus saddle points and plateaus, and which one you land in depends on the starting point.

and the *hidden error signal* as

$$\delta_i^h = \delta_i a \sigma'(u_i). \tag{2.7}$$

As the name suggests, the error signals will *signal* the neurons in which direction to change their parameters, and by how much. With all these new variables, the partials (2.4) read

$$\begin{aligned} \frac{\partial L}{\partial a} &= \sum_i \delta_i z_i \\ \frac{\partial L}{\partial b} &= \sum_i \delta_i \\ \frac{\partial L}{\partial c} &= \sum_i \delta_i^h x_i \\ \frac{\partial L}{\partial d} &= \sum_i \delta_i^h \end{aligned} \tag{2.8}$$

Phew! Much cleaner than (2.4)! And in return we've gained an astonishing behaviour: instead of computing all the messy partial derivatives in (2.4), all we need to do in order to calibrate each neuron's parameters is to compute its respective error signal and apply elementary algebra. This shortcut is called *backpropagation*. The name stems from the fact that, starting at the output and going backwards, each crossed layer will render the neuron one error signal. Look carefully: a and b each gained a δ_i plus z_i for a; similarly, c and b each gained a δ_i^h plus x_i for c. Were there a third layer, the recipe would repeat.

This behaviour, showed up explicitly in this two-neuron toy model, is in fact a general rule to networks of any size. This prepares our terrain for the next chapter.

Chapter 3

Multi-neuron ML

We now have all necessary ingredients to build a multi-neuron network. So, let's do it! The first step is to adapt the input layer to be able to receive, instead of a single scalar x, a more generic object. You may argue "the most generic object is an $N \times M$ matrix, or a tensor of arbitrary rank!" And you're right, as I thought I was the first time I philosophized about the subject. However... When you stop to think about it, you realize that any generic object, whether it is a matrix or a tensor, can be *flattened* to a vector of some dimension. This flattening process can always take place outside of the network, in a data preprocessing stage. So, as far as the network itself is concerned, the most general input object is an N-dimensional vector.

The above considerations lead us to define the architecture of the first layer as an N-neuron input layer. I will make use of tensor index notation to write the input parameter of (2.5) as

$$x^i_\nu \qquad \text{input} \tag{3.1}$$

I have promoted the i-index to the top (remember, this is the index labeling the ith data point) while using greek letters at the bottom as the vector indices. With this notation, the hidden pre-activation step reads

$$u^i_\mu = w^{(1)}_{\mu\nu} x^i_\nu + b^{(1)}_\mu \qquad \text{hidden pre-activation} \tag{3.2}$$

In (3.2), the weight c from (2.5) has turned into the $w^{(1)}_{\mu\nu}$ matrix (the letter w is the standard one for referring to the $w$eights; I've been using a, b, c and d out of laziness) whereas the bias parameter, called d in (2.5), has turned into a bias vector $b^{(1)}_\mu$. The $^{(1)}$ superscript indicates these are the weights and bias of the first layer; as we start composing several layers, this number will grow. One very important observation: the ν-index ranges from 1 to the dimension of the input vector; the μ-index ranges from 1 to the size of the hidden layer. The $w^{(1)}_{\mu\nu}$ matrix connects every input scalar to every neuron in the hidden layer.

We must now define the activation function for the neurons in the hidden layer. Following nomenclature in equation (2.5) and plugging layer index $^{(1)}$ and neuron index μ, it becomes:

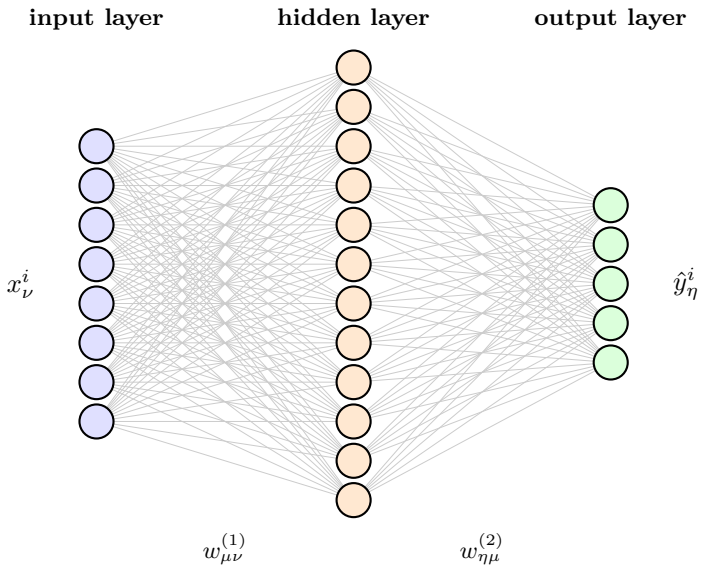
$$z^i_\mu = \sigma^{(1)}(u^i_\mu) \qquad \text{hidden activation} \tag{3.3}$$

There are several options on the market for implementing $\sigma^{(1)}$ as an actual function, all of which are listed in Appendix A. The standard choice for hidden layers is ReLU, although this may vary from one problem to another. For now, I will leave all options open so that, in the coming chapters, we can select them and build actual models that operate on specific, concrete data. That said, we can move on to the final step: defining the output layer. Let's carry out both steps at once:

$$\begin{aligned} v^i_\eta &= w^{(2)}_{\eta\mu} z^i_\mu + b^{(2)}_\eta \qquad \text{output pre-activation} \\ \hat{y}^i_\eta &= \sigma^{(2)}(v^i_\eta) \qquad \text{network output} \end{aligned} \tag{3.4}$$

A few words on (3.4) are worth saying. The index η runs over the number of neurons in the output layer. It can be one neuron with linear activation, if we are talking about a problem that must output one single unbounded number; it can be two neurons for selecting two binary, mutually exclusive classes; it can be three neurons firing simultaneously with probabilities; it can be a hundred neurons with whatever activation the problem happens to demand! Architecting the output layer is a real art that depends on the problem at hand. I will leave it open and build concrete examples in the next chapter. One last word in case this slipped under the radar: the matrix $w^{(2)}_{\eta\mu}$ belongs to the second layer and it connects every μ-neuron from the previous layer to every η-neuron in the output layer; final answer $\hat{y}^i_\eta$ is a vector of dimension η coming from the ith data input.

A picture is worth more than a thousand words, isn't it?



For the sake of completeness, let me write one last formula for the loss minimization condition. With all poetic license permission from mathematics, I'll write it this way:

$$\frac{\partial}{\partial(w,b)}L = 0 \tag{3.5}$$

So much is hidden behind (3.5), but since for us it is a symbolic equation, we need not worry. Modern frameworks like PyTorch, TensorFlow, scikit-learn and Keras take care of all the algebra. Just to give you an idea: a network with 8 input, 12 hidden and 5 output neurons has a total of 173 trainable w and b parameters, and therefore (3.5) leads to a system of 173 equations. Cool, huh? To be a bit more explicit, take a look at the index notation of the gradient descent equations:

$$\begin{aligned} w^{(\ell)}_{\mu\nu} &\to w^{(\ell)}_{\mu\nu} - \alpha\,\frac{\partial L}{\partial w^{(\ell)}_{\mu\nu}} \\ b^{(\ell)}_{\mu} &\to b^{(\ell)}_{\mu} - \alpha\,\frac{\partial L}{\partial b^{(\ell)}_{\mu}} \end{aligned} \tag{3.6}$$

All 173 updates of a hypothetical $(8, 12, 5)$ network are accounted for in these two lines. We could proceed by writing out the backpropagation formulas and seeing how equation (2.8) generalizes elegantly using index notation. However, let's save that for a later, more theoretical chapter, as I now want to work through some concrete examples!

Chapter 4

Concrete examples

Let's build our first model out of equations (3.1)—(3.4). For didactic purposes, I will use Keras to write this very first piece of code in the book:

```
import keras
from keras import layers

N_FEATURES = 8
N_HIDDEN = 12
N_CLASSES = 5

model = keras.Sequential(
    [
        keras.Input(shape=(N_FEATURES,)),
        layers.Dense(N_HIDDEN, activation="relu"),
        layers.Dense(N_CLASSES, activation="softmax")
    ]
)
```

The above implementation faithfully represents the network depicted in the figure of the preceding page: an input layer that receives vectors of dimension 8, followed by a hidden layer of 12 neurons with ReLU activation, and an output layer of 5 neurons with softmax activation.

The ReLU activation of the hidden layer, given by,

$$\sigma_{\text{ReLU}}(x) = \max(0, x) \tag{4.1}$$

is a fairly standard choice in many modern applications (it simply returns x if $x > 0$ and 0 otherwise) due to its low computational cost and empirically proven efficiency. But, what about softmax activation in the output layer? Why did I choose it?

Softmax activation, given by,

$$\hat{y}_\eta = \sigma_{\text{softmax}}(v_\eta) = \frac{e^{v_\eta}}{\sum_{j=1}^{5} e^{v_j}} \tag{4.2}$$

has one very powerful propertie: the output of all 5 neurons sums to one, that is, $\sum_{\eta=1}^{5} \hat{y}_\eta = 1$. Each output value $\hat{y}_\eta$, therefore, can be interpreted as the probability that input x_ν belongs to class η. Observe, therefore, that the choice of softmax activation in the output layer made this network suitable for handling classification problems: given a set of features x_ν, to which class η does this data point belong?

Two final ingredients are missing to complete the model: choosing the loss function and the learning rate. This can be achieved by the following piece of code:

```
model.compile(
    optimizer=keras.optimizers.SGD(learning_rate=0.01),
    loss=keras.losses.SparseCategoricalCrossentropy(),
    metrics=["accuracy"],
)
```

Take a moment to get acquainted to the nomenclature before we proceed to training of this model.

Chapter 5

Appendix A

- *ReLU* (Rectified Linear Unit), $\mathrm{ReLU}(x) = \max(0, x)$ — the default choice: trivially cheap to evaluate and it never saturates for $x > 0$, so the error signal survives many layers.
- *Leaky ReLU*, $\mathrm{LReLU}(x) = \max(\gamma x, x)$ with a small fixed γ (typically 0.01) — cures the "dead neuron" pathology of ReLU, whose derivative vanishes identically for negative inputs.
- *PReLU* (Parametric ReLU), same form as Leaky ReLU but with γ promoted to a trainable parameter — lets the network learn how leaky it wants to be, at the cost of one extra parameter per neuron (or per layer).
- *ELU* (Exponential Linear Unit),

$$\mathrm{ELU}(x) = \begin{cases} x, & x > 0 \\ \beta\,(e^x - 1), & x \le 0 \end{cases}$$

  — smooth everywhere and saturating at $-\beta$ for large negative inputs, which pulls the mean activation toward zero and speeds up convergence.
- *GELU* (Gaussian Error Linear Unit), $\mathrm{GELU}(x) = x\,\Phi(x) = \frac{x}{2}\left[1 + \mathrm{erf}\left(x/\sqrt{2}\right)\right]$, with Φ the standard normal cumulative distribution — a smooth, probabilistically motivated gate that weights each input by how likely it is to be positive; the standard in modern transformer architectures.
- *Sigmoid/Logistic*, $\sigma(x) = 1/(1 + e^{-x})$ — our old friend from chapter 2, squashing the real line into $(0, 1)$; ideal for probabilities at the output, but its derivative never exceeds $1/4$, which starves deep networks of gradient.
- *Tanh*, $\tanh(x) = (e^x - e^{-x})/(e^x + e^{-x})$ — a rescaled sigmoid mapping onto $(-1, 1)$, preferable to it because it is centred at zero, though it saturates just as badly.